Software Requirements Specification (SRS)

NileTech E-Learning Platform

Version 1.0

Prepared for: Interns

Prepared by: Nile Technology Solution

Document Approval

Name	Role	Signature	Date
Adem Kedir	Project Manager		March 20, 2025

1. Introduction

1.1 Purpose

This document defines the requirements for the **NileTech E-Learning Platform**, a globally accessible online learning system designed to deliver scalable education to students and professionals worldwide.

1.2 Scope

The NileTech platform will:

- Provide **course management** for instructors (upload PDFs, YouTube video links, quizzes).
- Enable **student enrollment** with payment options via Stripe or manual bank receipt verification.
- Include **quiz assessments** (multiple-choice, true/false) with question randomization.
- Offer **comment sections** for course discussions (no live chat).
- Operate as a **web-based platform** with mobile responsiveness.

1.3 Definitions, Acronyms, and Abbreviations

- **LMS**: Learning Management System
- **AWS**: Amazon Web Services
- **GDPR**: General Data Protection Regulation

1.4 References

- IEEE SRS Template (IEEE Std 830-1998).
- GDPR Compliance Standards.

1.5 Overview

This document outlines system requirements, user roles, and workflows for NileTech's mission to democratize education globally.

2. Overall Description

2.1 Product Perspective

The NileTech platform will:

- Integrate with **Stripe** for global payments and **YouTube** for video content.
- Support **manual bank receipt verification** for users without digital payment access.
- Operates as a standalone web application.

2.2 User Classes and Characteristics

1. **Students**: Enroll in courses, submit quizzes, view progress, and interact via comments.
2. **Instructors**: Create courses, upload PDFs/YouTube links, design quizzes, and moderate comments.
3. **Administrators**: Approve manual payments, manage users, and generate reports.
4. **Guest Users**: Browse course catalogs but cannot enroll without registration.

2.3 Operating Environment

- **Platforms**:
    - Web-based (Chrome, Firefox, Safari).
    - Mobile-responsive design (Android/iOS browsers).
- **Servers**:
    - Hosted on AWS (multi-region for global access).
    - Database: PostgreSQL for relational data, AWS S3 for PDF storage.

2.4 Design and Implementation Constraints

- **Technical Constraints**:
    - Web-only platform (no native mobile app in Phase 1).
- **Compliance Requirements**:
    - GDPR compliance for user data privacy.
    - Accessibility standards (WCAG 2.1).

2.5 Assumptions and Dependencies

- **Assumptions**:
    - Users have basic internet access.
- **Dependencies**:
    - Stripe API for payment processing.
    - YouTube Data API for video progress tracking.

3. System Features and Requirements

3.1 User Management

Functional Requirements:

- Sign-up with email and password.
- Role-based access control (student, instructor, admin).
- Password reset via email.

Non-Functional Requirements:

- Secure encryption (SSL/TLS, bcrypt for password hashing).

Data Flow:

- User data (email, password) → validated → stored in PostgreSQL.
- Roles enforced via middleware during login.

Technical Considerations:

- Use **Firebase Auth** or **Auth0** for authentication or any other.

3.2 Course Management

Functional Requirements:

- Instructors create courses with PDFs (max 50MB) and YouTube links.
- Set prerequisites (e.g., "Complete Course X to unlock Course Y").

Non-Functional Requirements:

- PDFs load in ≤3 seconds globally (optimized CDN delivery).

Data Flow:

- PDFs uploaded → stored in AWS S3.
- Course metadata (title, description, YouTube links) → stored in PostgreSQL.

Technical Considerations:

- Use **AWS CloudFront** for global PDF delivery (optional).

3.3 Content Delivery

Functional Requirements:

- Display embedded YouTube videos and PDFs via **PDF.js**.

- Track video progress using YouTube API.

Non-Functional Requirements:

- Mobile-responsive design.

Data Flow:

- YouTube API fetches video duration/watch time → updates progress in PostgreSQL.

Technical Considerations:

- Use **React Embed YouTube** library for video integration.

3.4 Assessments & Grading

Functional Requirements:

- Multiple-choice/true-false quizzes with **question randomization**.
- Auto-grading with instant results.

Non-Functional Requirements:

- Quizzes load in ≤5 seconds globally.

Data Flow:

- Quiz answers → compared to answer key → scores stored in PostgreSQL.

Technical Considerations:

- Randomize questions using **Fisher-Yates shuffle algorithm**.

3.5 Comment Section

Functional Requirements:

- Threaded comments under courses with reply notifications.
- Instructors can pin/delete comments.

Non-Functional Requirements:

- Notifications delivered via email within 1 minute.

Data Flow:

- Comments stored in PostgreSQL → notifications sent via **SendGrid** or **Nodemailer**.

Technical Considerations:

- Use **Server-Sent Events (SSE)** for real-time updates.

3.6 Payment System

Functional Requirements:

- **Stripe Integration**: Credit/debit card payments (multi-currency).
- **Manual Bank Receipts**: Students upload scanned receipts → admins verify.

Non-Functional Requirements:

- PCI DSS compliance for Stripe transactions.

Data Flow:

- Stripe payments → webhook updates payment status in PostgreSQL.
- Receipts stored in AWS S3 → admins approve via admin dashboard.

Technical Considerations:

- Use **Stripe Elements** for PCI-compliant payment forms.

3.7 Progress Tracking & Certificates

Functional Requirements:

- Student dashboards show course progress (quizzes, videos).
- Generate PDF certificates upon course completion.

Non-Functional Requirements:

- Certificates include dynamic user/course details.

Data Flow:

- Aggregated progress data → displayed via **Chart.js** graphs or any other.
- Certificates generated using **PDFKit**.

Technical Considerations:

- Use **i18next** for basic localization (if needed).

4. System Architecture & Data Flow

High-Level Architecture:

1. **Frontend**: React.js with responsive UI.
2. **Backend**: Node or Django REST API (Python) .
3. **Database**: PostgreSQL (relational) + MongoDB (analytics).
4. **Cloud**: AWS S3 + CloudFront (global content delivery).

Data Flow Diagram:

User → React Frontend → Node / Django API → PostgreSQL/MongoDB
↓
AWS S3 (PDFs) / Stripe/YouTube API

5. Knowledge Gaps & Mitigation

1. **Global Scalability**:
   - Use AWS CloudFront for CDN and load balancing.
2. **Email Notifications**:
   - Implement SendGrid for reliable email delivery.
3. **Payment Compliance**:
   - Adhere to Stripe's global compliance standards.

6. Prioritized Development Roadmap

Phase	Features	Tools
Phase 1	User auth, course creation, PDF hosting	Firebase, AWS S3, React.js
Phase 2	Quiz system, manual payment approval	Django, PostgreSQL, SendGrid
Phase 3	Certificates, progress dashboards	PDFKit, Chart.js